

Model Quantization

Team 06

Aditya Prakash Borate
23110065@iitgn.ac.in
23110065

Swayam Giridhar Borate
23110066@iitgn.ac.in
23110066

GITHUB REPO

1. Dataset Preparation:

- About Breast Cancer Dataset:
 - STT2 dataset is designed for sentiment analysis on spoken language data.
 - It contains speech samples labeled with sentiment categories.
- Python Code for Data Loading and Splitting:

```
!wget -O train.tsv  
"https://raw.githubusercontent.com/clairett/pytorch-  
sentiment-classification/master/data/SST2/train.tsv"  
!wget -O test.tsv  
"https://raw.githubusercontent.com/clairett/pytorch-  
sentiment-classification/master/data/SST2/test.tsv"  
  
# Load the dataset  
train_df = pd.read_csv("train.tsv", delimiter="\t",  
header=None, names=["Review", "Sentiment"])  
test_df = pd.read_csv("test.tsv", delimiter="\t",  
header=None, names=["Review", "Sentiment"])  
  
X_train = train_df['Review']  
y_train = train_df['Sentiment']  
X_test = test_df['Review']  
y_test = test_df['Sentiment']
```

```
# Split training data into train and validation (80-20 split)
X_train, X_val, y_train, y_val =
train_test_split(X_train, y_train, test_size=0.2,
random_state=42)
```

2. Construct a Multi-Layer Perceptron (MLP) model.

```
class MLPModel(nn.Module):
    def __init__(self, input_size=10000, hidden_sizes=[512, 256, 128,
64], output_size=2):
        super(MLPModel, self).__init__()

        # Define layers
        self.fc1 = nn.Linear(input_size, hidden_sizes[0])
        self.fc2 = nn.Linear(hidden_sizes[0], hidden_sizes[1])
        self.fc3 = nn.Linear(hidden_sizes[1], hidden_sizes[2])
        self.fc4 = nn.Linear(hidden_sizes[2], hidden_sizes[3])
        self.fc5 = nn.Linear(hidden_sizes[3], output_size)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3) # Dropout for regularization

    def forward(self, x):
        x = self.dropout(self.relu(self.fc1(x)))
        x = self.dropout(self.relu(self.fc2(x)))
        x = self.dropout(self.relu(self.fc3(x)))
        x = self.dropout(self.relu(self.fc4(x)))
        x = self.fc5(x)
        return x

# Initialize Model
model = MLPModel()

# Print Summary (batch_size=1, input_size=10000)
summary(model, input_size=(1, 10000), col_names=["input_size",
"output_size", "num_params", "trainable"])
```

Output:

Layer (type:depth-idx)	Input Shape	Output Shape	Param #	Trainable
MLPModel	[1, 10000]	[1, 2]	--	True
└─Linear: 1-1	[1, 10000]	[1, 512]	5,120,512	True
└─ReLU: 1-2	[1, 512]	[1, 512]	--	--
└─Dropout: 1-3	[1, 512]	[1, 512]	--	--
└─Linear: 1-4	[1, 512]	[1, 256]	131,328	True
└─ReLU: 1-5	[1, 256]	[1, 256]	--	--
└─Dropout: 1-6	[1, 256]	[1, 256]	--	--
└─Linear: 1-7	[1, 256]	[1, 128]	32,896	True
└─ReLU: 1-8	[1, 128]	[1, 128]	--	--
└─Dropout: 1-9	[1, 128]	[1, 128]	--	--
└─Linear: 1-10	[1, 128]	[1, 64]	8,256	True
└─ReLU: 1-11	[1, 64]	[1, 64]	--	--
└─Dropout: 1-12	[1, 64]	[1, 64]	--	--
└─Linear: 1-13	[1, 64]	[1, 2]	130	True
Total params: 5,293,122				
Trainable params: 5,293,122				
Non-trainable params: 0				
Total mult-adds (M): 5.29				

```
=====
Input size (MB): 0.04
Forward/backward pass size (MB): 0.01
Params size (MB): 21.17
Estimated Total Size (MB): 21.22
=====
```

3. Bag-of-words implementation for creating embeddings

```
vectorizer = CountVectorizer(max_features=10000)
X_train_bow = vectorizer.fit_transform(X_train).toarray()
X_val_bow = vectorizer.transform(X_val).toarray()
X_test_bow = vectorizer.transform(X_test).toarray()

# Convert labels to NumPy arrays before converting to PyTorch tensors
y_train = torch.tensor(y_train.to_numpy(), dtype=torch.long)
y_val = torch.tensor(y_val.to_numpy(), dtype=torch.long)
y_test = torch.tensor(y_test.to_numpy(), dtype=torch.long)
```

4. Model Training and Checkpointing

```
# Hyperparameters
HYPERPARAMS = {
    "input_size": 10000,
    "hidden_sizes": [512, 256, 128, 64],
    "output_size": 2,
    "epochs": 10,
    "batch_size": 32,
    "learning_rate": 0.0001
}

import torch
from torch.utils.data import DataLoader, TensorDataset
import matplotlib.pyplot as plt

def train_model(model, X_train, y_train, X_val, y_val,
                checkpoint_path):
    device = "cuda" if torch.cuda.is_available() else "cpu"
    model.to(device)

    optimizer = torch.optim.Adam(model.parameters(),
    lr=HYPERPARAMS["learning_rate"])
    criterion = torch.nn.CrossEntropyLoss()

    torch.cuda.empty_cache() # Free up memory

    train_loader = DataLoader(
        TensorDataset(torch.tensor(X_train, dtype=torch.float32),
        torch.tensor(y_train, dtype=torch.long)),
```

```
        batch_size=HYPERPARAMS["batch_size"], shuffle=True,
pin_memory=True
    )
    val_loader = DataLoader(
        TensorDataset(torch.tensor(X_val, dtype=torch.float32),
torch.tensor(y_val, dtype=torch.long)),
        batch_size=HYPERPARAMS["batch_size"], shuffle=False,
pin_memory=True
    )

    best_val_acc = 0.0
    best_model_state = None
    best_optimizer_state = None
    best_loss = None

    history = {
        "train_loss": [],
        "val_loss": [],
        "val_accuracy": []
    }

    for epoch in range(HYPERPARAMS["epochs"]):
        model.train()
        total_train_loss = 0

        for batch_X, batch_y in train_loader:
            batch_X, batch_y = batch_X.to(device), batch_y.to(device)

            optimizer.zero_grad()
            outputs = model(batch_X)
            loss = criterion(outputs, batch_y)
            loss.backward()
            optimizer.step()

            total_train_loss += loss.item()

        # Validation
        model.eval()
        total_val_loss = 0
        val_correct = 0
        val_total = 0
        with torch.no_grad():
            for batch_X, batch_y in val_loader:
                batch_X, batch_y = batch_X.to(device),
batch_y.to(device)

                outputs = model(batch_X)
                loss = criterion(outputs, batch_y)
```

```
        total_val_loss += loss.item()

        preds = torch.argmax(outputs, axis=1)
        val_correct += (preds == batch_y).sum().item()
        val_total += batch_y.size(0)

    val_acc = val_correct / val_total
    avg_train_loss = total_train_loss / len(train_loader)
    avg_val_loss = total_val_loss / len(val_loader)

    history["train_loss"].append(avg_train_loss)
    history["val_loss"].append(avg_val_loss)
    history["val_accuracy"].append(val_acc)

    print(f"Epoch {epoch+1}: Train Loss = {avg_train_loss:.4f},
Val Loss = {avg_val_loss:.4f}, Val Accuracy = {val_acc:.4f}")

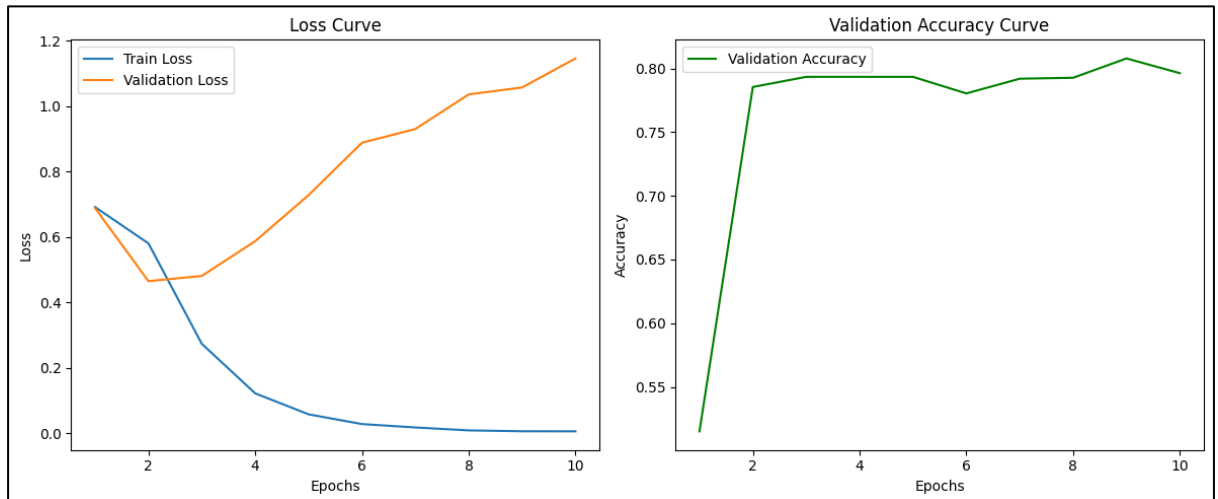
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        best_loss = avg_val_loss
        best_model_state = model.state_dict().copy()
        best_optimizer_state = optimizer.state_dict().copy()

    if best_model_state:
        checkpoint = {
            "model_state_dict": best_model_state,
            "optimizer_state_dict": best_optimizer_state,
            "hyperparameters": HYPERPARAMS,
            "performance": {
                "final_loss": best_loss,
                "final_accuracy": best_val_acc
            }
        }
        torch.save(checkpoint, checkpoint_path)
        print(f"Best model saved with Validation Accuracy:
{best_val_acc:.4f}")

    return model, history

# Initialize model
model_bow = MLPModel(input_size=10000)

# Train the model
trained_model_bow, history_bow = train_model(model_bow, X_train_bow,
y_train, X_val_bow, y_val, checkpoint_path="checkpoint")
```

Train-Val Loss/Validation Accuracy Curves**5. Quantization**

```
# Dynamic Quantization typically runs on CPU.
eval_device = torch.device("cpu")

def get_model_size(model, label=""):
    torch.save(model.state_dict(), "temp_model_state.pt")
    size_mb = os.path.getsize("temp_model_state.pt") / (1024 * 1024)
    os.remove("temp_model_state.pt")
    print(f"Size ({label}): {size_mb:.4f} MB")
    return size_mb

def evaluate_model(model, X_test, y_test, device,
input_dtype=torch.float32):
    model.to(device)
    model.eval()

    # Prepare DataLoader for evaluation
    test_loader = DataLoader(
        TensorDataset(torch.tensor(X_test, dtype=torch.float32), #
Input always starts as float32
                        torch.tensor(y_test, dtype=torch.long)),
        batch_size=128,
        shuffle=False
    )

    correct = 0
    total = 0
    total_inference_time = 0
    num_samples = 0

    with torch.no_grad():
        for batch_X, batch_y in test_loader:
```

```
dtype      # Move data to the evaluation device and cast to required
dtype
    batch_X = batch_X.to(device).to(input_dtype)
    batch_y = batch_y.to(device)

    start_time = time.perf_counter()

    # Perform inference
    outputs = model(batch_X)

    end_time = time.perf_counter()

    total_inference_time += (end_time - start_time)
    num_samples += batch_X.size(0)

    preds = torch.argmax(outputs, axis=1)
    correct += (preds == batch_y).sum().item()
    total += batch_y.size(0)

    accuracy = 100 * (correct / total)
    # Calculate average inference time per sample in milliseconds
    avg_inference_time_ms = (total_inference_time / num_samples) *
1000

    print(f"Accuracy: {accuracy:.2f}%")
    print(f"Average Inference Time: {avg_inference_time_ms:.4f}
ms/sample")

    return accuracy, avg_inference_time_ms
```

6. Original Model

```
# Load the Original Model
# Create a new instance of the model architecture
original_model = MLPModel(input_size=HYPERPARAMS["input_size"],
                           hidden_sizes=HYPERPARAMS["hidden_sizes"],
                           output_size=HYPERPARAMS["output_size"])

# Load the saved state dict from the checkpoint
checkpoint = torch.load("/kaggle/working/checkpoint",
map_location=eval_device) # Load directly to eval device
original_model.load_state_dict(checkpoint['model_state_dict'])
original_model.to(eval_device)
original_model.eval() # Set to evaluation mode

print("Original model loaded successfully.")
```

7. Dynamic Quantization

```
# Dynamic quantization modifies the model in place, so working on a
copy.
model_to_quantize = copy.deepcopy(original_model).to("cpu")
model_to_quantize.eval()

# Apply dynamic quantization
model_dynamic_quantized = torch.quantization.quantize_dynamic(
    model=model_to_quantize,
    qconfig_spec={nn.Linear}, # Quantize Linear layers to qint8
    dtype=torch.qint8
)

print("Dynamic quantization applied.")
print(model_dynamic_quantized)
```

8. Half Precision

```
# Create a copy and convert to half precision
model_half = copy.deepcopy(original_model).to(eval_device)
model_half.eval()
model_half = model_half.half() # Convert model parameters and buffers
to FP16

print("Half precision conversion applied.")
```

Final Table:

S.I.	Model Name	Accuracy (Out of 100)	Storage (In MB)	Inference time (In ms)
1	Original	79.2422	20.1953	0.0879
2	Dynamic	79.3520	5.0574	0.0621
3	Half	79.2422	10.0995	6.9205
